What is the problem?

Sometimes in **machine learning classification**, we have **imbalanced data** – one class has **many more samples** than the other.

For example:

Email Type	Number of Emails
Not Spam (0)	9,500
Spam (1)	500

This means your model will learn more about "Not Spam" and ignore "Spam" – which is **bad**.

---

1. Oversampling (Add More of the Minority Class)

🧠 Idea:

Add **more examples** of the **minority class** (in this case, "Spam") to **balance** the data.

✅ When to Use:

- When you have **very few** examples of one class.
- You **don't want to lose** any data.
- Dataset is **small**.

🎯 Example:

- You duplicate or synthesize more "Spam" emails.
- Now you have:
  - Not Spam = 9500
  - Spam = 9500

Now the model will treat both equally.

---

2. Undersampling (Remove Some of the Majority Class)

🧠 Idea:

Remove some examples from the **majority class** (in this case, "Not Spam") to make it equal.

✅ When to Use:

- When you have a **large dataset**.
- You are okay with **losing some data** to save time.
- Training is **slow** due to too much data.

🎯 Example:

- Randomly remove "Not Spam" emails.
- Now you have:
  - Not Spam = 500
  - Spam = 500

Again, the model will treat both equally.

🧠 Final Trick to Remember:

"Oversampling = Add more to the weaker side"
"Undersampling = Cut down the stronger side"

Code:

```
train_data_cata_encoded = pd.get_dummies(train_data_cat,
columns=train_data_cat.columns.to_list())
train_data_cata_encoded.head()
```

📌 What does this do?

This code is converting **categorical data** (text like "Male", "Yes", "Private", etc.) into **numeric format** (0s and 1s) so that a machine learning model can understand and use it.

Step-by-Step Explanation

◇ 1. train_data_cat

This is your original **categorical dataset**, like:

gender	ever_married	work_type	Residence_type	smoking_status
Male	Yes	Private	Urban	never smoked
Female	Yes	Self-employed	Rural	smokes
Male	Yes	Private	Urban	never smoked
Female	Yes	Private	Rural	smokes
Female	Yes	Self-employed	Urban	never smoked

These are all **text categories.**

◇ 2. pd.get_dummies(...)

This function:

- **Converts** each category into **separate columns.**
- Fills them with **1s and 0s:**
  - 1 = present
  - 0 = not present

This process is called **One-Hot Encoding.**

◇ 3. columns=train_data_cat.columns.to_list()

This tells pandas:

"Apply one-hot encoding to **all columns** in the dataset."

📜 Result:

gender_Female	gender_Male	gender_Other	ever_married_No	ever_married_Yes	...
0	1	0	0	1	...
1	0	0	0	1	...

So now instead of a single "gender" column, you have **3 columns**:

- gender_Female, gender_Male, gender_Other — only one of them will be 1, rest will be 0.

☑ Why is this useful?

Most machine learning models **don't understand text** like "Male" or "Private".
They **need numbers**.

So one-hot encoding helps by converting categories into **machine-readable format**.

🔁 Final Summary:

This code takes your **categorical columns**, breaks them into **separate binary columns**, and replaces the text with **0/1** values so that machine learning algorithms can use them properly.

Code:

```
data = pd.concat([train_data_cata_encoded, train_data_num], axis=1, join="outer")
data.head()
```

◇ What's happening here?

You are **combining two types of data** into one final dataset:

1. **train_data_cata_encoded** → This is your **categorical data,** already converted into 0s and 1s using get_dummies().
2. **train_data_num** → This is your **numerical data,** like:
    - age
    - bmi
    - avg_glucose_level
    - hypertension
    - heart_disease
    - stroke (target variable)

Explanation of each part:

☑ pd.concat([...])

This joins multiple DataFrames **side by side** or **on top of each other**.

- You gave a list: [train_data_cata_encoded, train_data_num]
- These will be **joined side by side** (horizontally), because:

☑ axis=1

- axis=1 means join **columns** (not rows).

☑ join="outer"

- Ensures that **no data is lost;** if there is any missing column between them, it will still keep all columns (usually not a problem if both DataFrames have same row indices).

☑ data.head()

This shows the **first 5 rows** of your new combined dataset.

📦 Final Dataset: data

Now you have **one full dataset** with:

- 🔸 One-hot encoded categorical features (e.g. gender_Female, work_type_Private, etc.)
- 🔹 Numeric features (e.g. age, bmi, avg_glucose_level, etc.)
- 🎯 The label (probably stroke, which is 0 or 1)

This is now **ready to be used** in a machine learning model.

🧠 Final Summary:

You combined **encoded categorical data** and **numerical data** into **one complete dataset** that a machine learning algorithm can understand.

Code:

```
lr = LogisticRegression(max_iter=200)
lr.fit(X_train, y_train)
y_pred1 = lr.predict(X_test)
print(accuracy_score(y_test, y_pred1))
accuracy[str(lr)] = accuracy_score(y_test, y_pred1)*100
```

◇ Step-by-Step Explanation:

1️⃣ lr = LogisticRegression(max_iter=200)

- You're creating a **Logistic Regression model**.
- `max_iter=200` means it will try up to 200 steps (iterations) to find the best model.
- This is useful if the model doesn't converge with the default (100) iterations.

2️⃣ lr.fit(X_train, y_train)

- This is where the **model learns from the training data**.
- `X_train` = your features (input data).
- `y_train` = labels (target values, like `stroke` = 0 or 1).

So here, you're teaching the model what patterns lead to a stroke or not.

3️⃣ y_pred1 = lr.predict(X_test)

- After training, now you're using the model to **predict** results on **unseen test data**.
- This returns a list of predicted values (like `[0, 1, 0, 0, 1]`).

4️⃣ print(accuracy_score(y_test, y_pred1))

- You're calculating how accurate your predictions are by comparing them to the **true labels (`y_test`)**.
- `accuracy_score` gives the percentage of correct predictions.

5️⃣ accuracy[str(lr)] = accuracy_score(y_test, y_pred1) * 100

- You're saving the **accuracy percentage** (like 92.5) in a dictionary called `accuracy`.
- `str(lr)` makes the model name a string (e.g., `'LogisticRegression(max_iter=200)'`), so it can be used as a key.

Final Summary:

You're training a Logistic Regression model, predicting results on test data, printing the accuracy, and storing it in a dictionary for later comparison.

Is the Input Layer Hidden?

Yes and No.

✅ In Keras, the input layer can be implicitly included in the first `Dense` layer using `input_shape`.

In your code:

```
keras.layers.Dense(4800, input_shape=[21], activation='relu')
```

This line:

- Defines the **first hidden layer** with **4800 neurons**
- Also tells Keras that the **input will have 21 features**

So this acts as both:

- **Input layer (shape info)** ✅
- **First hidden layer** ✅

That's why there's **no need to separately write**:

```
keras.Input(shape=(21,))
```

Unless you want more control (e.g., for functional API or custom inputs), Keras handles it automatically inside `Sequential()`.

So Why This Code Is OK?

```
model = keras.Sequential([
    keras.layers.Dense(4800, input_shape=[21], activation='relu'),
    ...
])
```

✔ Because:

- The **first Dense layer** is doing **two jobs**:
    1. **Receiving input of size 21**
    2. Acting as the **first hidden layer**

What's actually happening under the hood:

- Input: 21 features (like age, gender, BMI, etc.)
- Dense(4800): Connects each of those 21 inputs to **4800 neurons** with weights and biases.
- Then it passes to the next hidden layers.

Summary:

You don't need to write a separate input layer in Keras `Sequential` model.
If you provide `input_shape` in the first layer, Keras knows the shape of the input.
The input layer is there **behind the scenes**, it's just **not shown separately**.